

INFORMATION TECHNOLOGY PROJECT REPORT

An End-to-End Encrypted (E2EE) Messaging Simulator
and Cyber Threat Analysis Platform

Project: Secure Messenger

Development Team:

Dinh Ky Vi (Team Leader)

Nguyen Ngoc Hong Dao (Member - UI/UX & Frontend)

Nguyen Duy Bao Tran (Member - PoC Developer)

Nguyen Bui Minh Hang (Member - Security Specialist)

Nguyen Thi Sang, Dang Hong Nguyet

Hoang Thi Kim Ngân, Nguyen Thanh Huyen, Nguyen Phan Thanh Lich

July 2026

Abstract

This report presents the design and implementation of the **Secure Messenger** project – a web application that simulates a real-time End-to-End Encrypted (E2EE) messaging system. Developed primarily for educational purposes, the project helps students and security researchers gain a deeper understanding of modern cryptographic protocols such as the Diffie-Hellman Key Exchange, AES-256 symmetric encryption, and the Double Ratchet key rotation mechanism. Furthermore, the application integrates a simulated third-party threat scenario (Eve’s Hacker Console) to visually demonstrate the dangers of Cross-Site Scripting (XSS) and session hijacking/account takeover. The entire communication payload is synchronized in real-time using Firebase Realtime Database.

Contents

1	Introduction	3
1.1	Project Background	3
1.2	Project Objectives	3
2	Cryptographic Architecture and Core Mechanisms	3
2.1	Diffie-Hellman Key Exchange (DH-256)	3
2.2	Double Ratchet Mechanism	4
2.3	AES-256 Symmetric Encryption	4
2.4	SHA-256 Integrity Verification	4
3	User Interface and Technical Monitoring	4
3.1	Three-Column Layout	4
3.2	Ratchet Monitor Dashboard	4
4	Cyber Threat Simulation Scenario	4
4.1	XSS Exploitation Chain	5
4.2	Phishing Modal and Session Hijacking	5
5	Tech Stack and Contribution Roles	5
5.1	Technology Stack	5
5.2	Team Contribution Roles	5
6	Conclusion and Future Work	5
6.1	Project Outcomes	5
6.2	Future Directions	6

1 Introduction

1.1 Project Background

In the digital age, securing communication channels is a paramount requirement for both individuals and organizations. End-to-End Encryption (E2EE) has emerged as the gold standard for popular messaging applications like Signal, WhatsApp, and Telegram. However, for most general users and computer science students, the underlying mathematical structures and data processing pipelines of E2EE remain highly abstract.

1.2 Project Objectives

The **Secure Messenger** project was established to address the following key objectives:

- Visualize the mathematical mechanics of cryptography (specifically Diffie-Hellman and Double Ratchet) through real-time charts and data chips.
- Provide a secure testing environment (sandbox) to experience a realistic cyber threat scenario.
- Develop a fully functional Web application with a fluid Vanilla CSS/JS frontend integrated with a Cloud Database (Firebase) for message transport.

—

2 Cryptographic Architecture and Core Mechanisms

The cryptographic framework of Secure Messenger relies on a tight integration of asymmetric key exchange and symmetric key encryption algorithms.

2.1 Diffie-Hellman Key Exchange (DH-256)

The Diffie-Hellman key exchange protocol allows two parties to establish a shared secret key over an insecure communication channel without prior knowledge of each other. The default parameters in this simulation are a prime number $P = 23$ and a generator $G = 5$.

1. Alice selects a random private integer X_A (e.g., $X_A = 6$) and computes her public key:

$$Y_A = G^{X_A} \pmod{P} = 5^6 \pmod{23} = 8 \quad (1)$$

2. Bob selects a random private integer X_B (e.g., $X_B = 15$) and computes his public key:

$$Y_B = G^{X_B} \pmod{P} = 5^{15} \pmod{23} = 19 \quad (2)$$

3. The Shared Secret Key K is computed independently by both parties:

$$K = Y_B^{X_A} \pmod{P} = 19^6 \pmod{23} = 2 \quad (3)$$

$$K = Y_A^{X_B} \pmod{P} = 8^{15} \pmod{23} = 2 \quad (4)$$

This shared secret key K is then passed as the key for symmetric block cipher encryption.

2.2 Double Ratchet Mechanism

Simulating the Double Ratchet protocol (modeled after the Signal Protocol), the application automatically rotates the session keys upon sending or receiving messages. This guarantees Perfect Forward Secrecy (PFS) – ensuring that even if an attacker compromises a current session key, they cannot decrypt past messages or future traffic.

2.3 AES-256 Symmetric Encryption

Upon key generation or rotation, message payloads are encrypted using the Advanced Encryption Standard (AES) with a 256-bit key length. The application uses the CBC (Cipher Block Chaining) mode and PKCS7 padding via the `CryptoJS` library. The Initialization Vector (IV) is synchronized to ensure accurate decryption at the receiver end.

2.4 SHA-256 Integrity Verification

To prevent Man-in-the-Middle (MitM) tampering, the sender computes the SHA-256 hash of the ciphertext prior to transmission:

$$\text{Hash} = \text{SHA256}(\text{Ciphertext}) \quad (5)$$

The receiver computes the hash of the received ciphertext and compares it with the sent hash. If they match, the ciphertext is confirmed to be untampered and is decrypted; otherwise, the UI flags a data manipulation warning.

3 User Interface and Technical Monitoring

3.1 Three-Column Layout

The Secure Messenger interface is built natively using HTML5 and Vanilla CSS, optimizing layout space to show:

- **Left Column (Alice’s View):** Chat window for Alice.
- **Middle Column (Ratchet Monitor):** Real-time visualization of key derivations, session keys, SHA-256 hashes, and database payload schema.
- **Right Column (Bob’s View):** Chat window for Bob.

3.2 Ratchet Monitor Dashboard

The monitor provides instant feedback on cryptographic state transitions. Glowing glass-morphic chips reflect changes in real-time, helping users correlate UI actions with low-level cryptographic operations.

4 Cyber Threat Simulation Scenario

A defining feature of the application is the integration of the **Hacker Console** under the persona of the eavesdropper, Eve.

4.1 XSS Exploitation Chain

The simulated Cross-Site Scripting (XSS) attack runs as follows:

1. **Malicious Link:** Eve sends a phishing link to Alice.
2. **Payload Execution:** Alice clicks the link, triggering DOM injection.
3. **Remote Cursor Control:** A fake mouse cursor appears on Alice’s screen and moves autonomously, simulating a remote desktop compromise.

4.2 Phishing Modal and Session Hijacking

Following the cursor takeover, a fake security verification dialog (*Fake Login Modal*) overlays Alice’s workspace. Once Alice enters her credentials:

- The console transmits the password to Eve’s hacker interface.
 - Eve’s command-line interface (styled with a green matrix terminal font) updates to show the successful compromise and flags an *Account Takeover Success*.
-

5 Tech Stack and Contribution Roles

5.1 Technology Stack

The application relies on lightweight, standard web libraries to maintain near-instant loading times:

- **Frontend:** HTML5, CSS3, JavaScript ES Modules.
- **Cryptography:** CryptoJS v4.1.1.
- **Onboarding:** Driver.js v1.0.1 (interactive tour guide).
- **Database:** Firebase Realtime Database.

5.2 Team Contribution Roles

6 Conclusion and Future Work

6.1 Project Outcomes

The **Secure Messenger** project successfully visualizes End-to-End Encryption and highlights web security vulnerabilities via simulated XSS execution. The tool is highly effective as an educational utility for showing how modern transport channels can be secured mathematically yet compromised through user-facing attack vectors.

Table 1: Task Allocation Matrix

Member	Core Role	Key Contributions
Dinh Ky Vi	Project Leader	Core crypto algorithms, E2EE data flows
Nguyen Ngoc Hong Dao	UI/UX & Frontend	Interface design, transitions, and translation
Nguyen Duy Bao Tran	Integrator & PoC	Java POC simulation, Firebase sync wiring
Nguyen Bui Minh Hang	Security Specialist	XSS attack scenario and Hacker Console
Support Group	QA & Copywriting	Test cases, slides, and documentation

6.2 Future Directions

Future expansions for the platform include:

- Allowing users to customize Diffie-Hellman group parameters (e.g., DH-512 or DH-1024).
- Adding MitM intercept nodes in the network flow visualization.
- Implementing WebRTC for actual browser-to-browser P2P communication.

References

- [1] W. Diffie and M. Hellman, “New directions in cryptography,” in *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644-654, November 1976.
- [2] National Institute of Standards and Technology (NIST), “FIPS PUB 197: Advanced Encryption Standard (AES),” November 2001.
- [3] T. Perrin and D. Whisper Systems, “The Double Ratchet Algorithm,” 2016. [Online]. Available: <https://signal.org/docs/specifications/doubleratchet/>